

2026-05-08-p4-test-hardening

Phase 4 — Test/Challenge Hardening — Implementation Plan

For agentic workers: REQUIRED SUB-SKILL: Use superpowers:subagent-driven-development or superpowers:executing-plans. Steps use checkbox (- []) syntax for tracking.

Goal: Every test and Challenge carries positive runtime evidence. Zero false-success results. All `t.Skip()` have SKIP-OK: markers. New anti-bluff verifier challenge.

Architecture: Audit → classify → fix. Each bluff type (wrapper/contract/structural/comment/skip) has a specific detection and remediation pattern. New `anti_bluff_verifier` challenge gate-keeps all future changes.

Tech Stack: Go testing, testify, bash, sha256sum

Spec: docs/superpowers/specs/2026-05-08-helixcode-zero-bluff-completion-design.md

File Structure Map

docs/improvements/test_audit_p4.md	– create (audit report)
tests/e2e/challenges/anti_bluff_verifier/challenge.go	– create
tests/e2e/challenges/anti_bluff_verifier/expected.json	– create
helix_code/**/*_test.go	– modify (fix identi

Task P4-T01: Full test suite audit

Files: - Create: docs/improvements/test_audit_p4.md



Step 1: Run full test suite, capture output

```
cd /run/media/milosvasic/DATA4TB/Projects/helix_code/HelixCode
go test -v ./... -count=1 2>&1 | tee /tmp/full_test_output.txt
```



Step 2: Classify every test

```
cd /run/media/milosvasic/DATA4TB/Projects/helix_code/HelixCode
cat > /tmp/audit.sh << 'EOF'
#!/bin/bash
echo "# Test Suite Audit Report" > docs/improvements/
test_audit_p4.md
```

```

echo "## $(date)" >> docs/improvements/test_audit_p4.md
echo "" >> docs/improvements/test_audit_p4.md

echo "### Skip Bluffs (no SKIP-OK marker)" >> docs/improvements/
    test_audit_p4.md
grep -rn 't\.Skip(' --include="*_test.go" . | grep -v "SKIP-OK" |
    while read line; do
        echo "- BAD: $line" >> docs/improvements/test_audit_p4.md
done

echo "" >> docs/improvements/test_audit_p4.md
echo "### Silent Passes (tests with zero assertions)" >> docs/
    improvements/test_audit_p4.md
grep -rn 'func Test' --include="*_test.go" . -l | while read f;
do
    count=$(grep -c 'assert\\.\\|require\\.\\|t\\.Error\\|t\\.Fatal' "$f"
        2>/dev/null || echo 0)
    testCount=$(grep -c 'func Test' "$f" 2>/dev/null || echo 0)
    if [ "$count" -lt "$testCount" ]; then
        echo "- SUSPECT: $f — $testCount test funcs, $count
            assertions" >> docs/improvements/test_audit_p4.md
    fi
done

echo "" >> docs/improvements/test_audit_p4.md
echo "### Integration/Challenge Files With Mocks" >> docs/
    improvements/test_audit_p4.md
grep -rn "testify/mock\\|\\.On\\" --include="*_test.go" . | grep -v
    "/internal/" | while read line; do
        echo "- BAD (mock in non-unit test): $line" >> docs/
            improvements/test_audit_p4.md
done
EOF
bash /tmp/audit.sh

```



Step 3: Review audit report

```
cat docs/improvements/test_audit_p4.md
```

Note all issues that need fixing.



Step 4: Commit audit

```

cd /run/media/milosvasic/DATA4TB/Projects/HelixCode
git add docs/improvements/test_audit_p4.md
git commit -m "docs(P4-T01): full test suite audit — skip/silent/
    mock patterns

```

Task P4-T02: Challenge harness audit



Step 1: Audit challenge runner for exit-code bugs (wrapper bluff)

```
cd /run/media/milosvasic/DATA4TB/Projects/helix_code/helix_code/
tests/e2e/challenges
grep -rn "os.Exit\\|exitCode\\|exit code\\|PASS.*but\\|FAIL.*but" --
include="*.go" . | head -20
```

Check the runner's exit-code logic. The pattern to verify:

```
if all assertions pass AND runtime evidence collected → exit 0
if any assertion fails → exit non-zero
```



Step 2: Audit expected.json for real assertions

```
for f in $(find . -name "expected.json"); do
  if grep -q "PASS\\|SUCCESS\\|OK" "$f" && ! grep -q "sha256\\|
checksum\\|runtime" "$f"; then
    echo "CONCERN (pass-by-absence): $f"
  fi
done
```

Expected: zero CONCERN entries (every challenge has positive evidence assertions)



Step 3: Document findings in audit report

```
cat >> docs/improvements/test_audit_p4.md << 'EOF'

## Challenge Harness Audit

### Exit-Code Logic
[audit result]

### expected.json Assertions
[audit result]
EOF
cd /run/media/milosvasic/DATA4TB/Projects/HelixCode
git add docs/improvements/test_audit_p4.md
git commit -m "chore(P4-T02): audit challenge harness for
wrapper/contract bluffs"
```

Task P4-T03: Bluff taxonomy sweep



Step 1: Automated bluff pattern scan

```
cd /run/media/milosvasic/DATA4TB/Projects/helix_code/HelixCode
echo "=== Wrapper Bluff: exit-code masking ==="
grep -rn "os.Exit(0)" --include="*_test.go" . && echo "warn:
    tests should not os.Exit(0)"
echo "=== Comment Bluff: comment vs code mismatch ==="
grep -rn "should\\|must\\|will" --include="*.go" internal/ cmd/ |
    grep "// " | grep -v "test\\|doc" | head -10
echo "=== Structural Bluff: committed but broken ==="
find . -name "*.go" -not -name "*_test.go" -exec grep -l "FIXME\\|
    BROKEN\\|DOES NOT WORK" {} \; | head -10
```



Step 2: Document findings, commit

```
cat >> helix_code/docs/improvements/test_audit_p4.md << 'EOF'

## Bluff Taxonomy Sweep
- Wrapper: [findings]
- Contract: [findings]
- Structural: [findings]
- Comment: [findings]
- Skip: [findings]
EOF
cd /run/media/milosvasic/DATA4TB/Projects/HelixCode
git add docs/improvements/test_audit_p4.md
git commit -m "chore(P4-T03): bluff taxonomy sweep across test
    suite

Phase: 4   Task: P4-T03"
```

Task P4-T04: Fix all identified bluffs



Step 1: Fix skip bluffs — Add SKIP-OK: #<ticket> to every bare t.Skip()

```
cd /run/media/milosvasic/DATA4TB/Projects/helix_code/HelixCode
# For each skip without marker, add: t.Skip("SKIP-OK: #<ticket>
    <reason>")
```



Step 2: Fix silent passes — Add real assertions to tests with zero assertions



Step 3: Fix mock usage in non-unit tests — Replace mocks with real services



Step 4: Fix exit-code bugs in challenge runner

If the runner returns 0 even when assertions fail, fix it.



Step 5: Verify fixes: re-run audit

Repeat P4-T01 audit, confirm zero issues



Step 6: Commit all fixes

```
cd /run/media/milosvasic/DATA4TB/Projects/HelixCode
git add helix_code/
git commit -m
    "fix(P4-T04): fix all identified test bluffs – skip
    markers, assertions, mocks"
```

Phase: 4 Task: P4-T04"

Task P4-T05: Anti-bluff verifier challenge

Files: - Create: helix_code/tests/e2e/challenges/anti_bluff_verifier/challenge.go - Create: helix_code/tests/e2e/challenges/anti_bluff_verifier/expected.json



Step 1: Create the challenge

```
package main

import (
    "crypto/sha256"
    "encoding/hex"
    "fmt"
    "os"
    "os/exec"
    "strings"
)

func main() {
    patterns := []string{
        "simulated", "placeholder", "stub", "TODO", "FIXME",
    }
    root := "../../../.."
    pass := true
    evidence := make(map[string]string)
```

```

for _, p := range patterns {
    cmd := exec.Command("grep", "-rn", p, "--include=*.go",
        "--exclude=*_test.go", "--exclude=*.bak*", "--exclude-
        dir=vendor", root)
    out, _ := cmd.Output()
    if len(out) > 0 {
        fmt.Printf("BLUFF FOUND (%s):\n%s\n", p, string(out))
        pass = false
    } else {
        fmt.Printf("PASS: no '%s' in production code\n", p)
    }
    evidence[p] = hex.EncodeToString(sha256.New().Sum(out))
}

if pass {
    fmt.Println("ANTI-BLUFF: ALL CLEAN")
    os.Exit(0)
}
fmt.Println("ANTI-BLUFF: FAILED")
os.Exit(1)
}

```



Step 2: Create expected.json

```

{
    "expected_result": "PASS",
    "evidence_type": "sha256",
    "bluff_patterns_checked": ["simulated", "placeholder", "stub",
        "TODO", "FIXME"],
    "skip_markers_checked": true,
    "runtime_evidence_required": true
}

```



Step 3: Run challenge, verify it passes

```

cd helix_code/tests/e2e/challenges/anti_bluff_verifier && go run
    challenge.go

```

Expected: ANTI-BLUFF: ALL CLEAN



Step 4: Commit

```

cd /run/media/milosvasic/DATA4TB/Projects/HelixCode
git add helix_code/tests/e2e/challenges/anti_bluff_verifier/
git commit -m "feat(P4-T05): add anti_bluff_verifier challenge
    gate

```

Task P4-T06: Fill challenge coverage gaps



Step 1: Identify packages lacking challenges

```
cd /run/media/milosvasic/DATA4TB/Projects/helix_code/HelixCode
for pkg in $(go list ./internal/... | grep -v test | grep -v
    mock); do
    pkgPath=$(echo $pkg | sed 's|dev.helix.code/||')
    if [ ! -d "tests/e2e/challenges/${pkgPath//\//_}" ]; then
        echo "MISSING CHALLENGE: $pkg"
    fi
done
```



Step 2: Create challenges for each missing package

For each package without a challenge: 1. Create challenge directory with `challenge.go` 2. Create `expected.json` with positive evidence assertions 3. Write the challenge to exercise the package's core functionality 4. Ensure it produces sha-256 evidence or equivalent



Step 3: Run all new challenges, verify they pass

```
for challenge in tests/e2e/challenges/*; do
    [ -f "$challenge/expected.json" ] || continue
    echo "Running: $challenge"
    (cd "$challenge" && go run challenge.go) && echo "PASS" ||
        echo "FAIL"
done
```



Step 4: Commit

```
cd /run/media/milosvasic/DATA4TB/Projects/HelixCode
git add helix_code/tests/e2e/challenges/
git commit -m
    "feat(P4-T06): fill challenge coverage gaps for all
    internal packages"
```

Task P4-T07: Full infrastructure test run



Step 1: Start full test infrastructure

```
cd /run/media/milosvasic/DATA4TB/Projects/helix_code/HelixCode
make test-infra-up
```



Step 2: Run complete test suite

```
make test-complete
```

Expected: zero failures (credential-gated SKIP-OK allowed)



Step 3: Tear down

```
make test-infra-down
```



Step 4: Document results

```
cat > helix_code/docs/improvements/full_infra_test_results_p4.md
    << 'EOF'
# Full Infrastructure Test Results (P4-T07)
Date: $(date)
Command: make test-complete
Result: [PASS/FAIL]
Skip markers: [count SKIP-OK]
Failures: [count]
EOF
cd /run/media/milosvasic/DATA4TB/Projects/HelixCode
git add docs/improvements/full_infra_test_results_p4.md
git commit -m "docs(P4-T07): full infrastructure test run results

Phase: 4  Task: P4-T07"
```

Task P4-T08: Cross-compile verification



Step 1: Build for all target platforms

```
cd /run/media/milosvasic/DATA4TB/Projects/helix_code/HelixCode
GOOS=linux GOARCH=amd64 go build ./... && echo "linux/amd64:
    PASS" || echo "linux/amd64: FAIL"
GOOS=darwin GOARCH=arm64 go build ./... && echo "darwin/arm64:
    PASS" || echo "darwin/arm64: FAIL"
GOOS=windows GOARCH=amd64 go build ./... && echo "windows/amd64:
    PASS" || echo "windows/amd64: FAIL"
```

Expected: PASS for all three



Step 2: Commit Phase 4 close-out

```
cd /run/media/milosvasic/DATA4TB/Projects/HelixCode
git add .
git commit -m "chore(P4-T08): Phase 4 complete – cross-compile
              verified, infrastructure tests pass"
```

Phase: 4 Task: P4-T08"

```
git push github main
```

Phase 4 Completion Checklist



Test audit complete, all bluffs classified



All skip bluffs fixed (SKIP-OK markers)



All silent-pass tests have real assertions



Challenge runner exit-code logic verified correct



Anti-bluff verifier challenge passes (ALL CLEAN)



Challenge coverage at 100% for internal packages



Full infrastructure test run passes



Cross-compile for linux/darwin/windows passes



Continue to Phase 5